

机器学习基础作业3

宋思逸

2400010816

2026 年 4 月 3 日

4.9

我们的核心思想是：基于已知特征值的样本来估计属性的重要性，并以概率形式处理缺失样本的去向。

假设训练集为 $\mathcal{D}$ ，离散属性 a 有 V 个可能的取值 $\{a^1, \dots, a^V\}$ 令 $\tilde{\mathcal{D}}$ 表示在属性 a 上没有缺失值的样本子集。

对于每一个样本 $\mathbf{x}$ ，我们赋予一个权重 w_x 。初始权重设为1。

定义无缺失值样本占比

$$\rho = \frac{\sum_{\mathbf{x} \in \tilde{\mathcal{D}}} w_x}{\sum_{\mathbf{x} \in \mathcal{D}} w_x} \quad (1)$$

无缺失值样本中，属于第 k 类样本的比例

$$\tilde{p}_k = \frac{\sum_{\mathbf{x} \in \tilde{\mathcal{D}}} w_x}{\sum_{\mathbf{x} \in \mathcal{D}} w_x} \quad (2)$$

无缺失值样本中取值为 a^v 的样本比例

$$\tilde{r}_v = \frac{\sum_{\mathbf{x} \in \tilde{\mathcal{D}}^v} w_x}{\sum_{\mathbf{x} \in \mathcal{D}} w_x} \quad (3)$$

基于 $\tilde{\mathcal{D}}$ ，计算纯度

$$\text{Gini}(\tilde{\mathcal{D}}) = 1 - \sum_{k=1}^{|Y|} \tilde{p}_k^2 \quad (4)$$

计算基尼指数时，我们只利用无缺失值的样本信息，并根据无缺失值样本所占比例加权

$$\text{Gini_index}(\mathcal{D}, a) = \rho \sum_{v=1}^V \tilde{r}_v \text{Gini}(\tilde{\mathcal{D}}^v) \quad (5)$$

从而在样本划分时，选定 a 作为划分属性之后，对于数据集 $\mathcal{D}$ 中的样本 $\mathbf{x}$ ，如果样本 $\mathbf{x}$ 在属性 a 上的取值已知为 a^v ，那么直接将该样本划入子节点 D^v ，并保持样本权重 w_x 不变；若样本 $\mathbf{x}$ 在属性 a 上的取值缺失，则将该样本同时划入所有子节点，但调整其权重

$$w'_x = w_x \tilde{r}_v \quad (6)$$

从而实现了同一个样本以不同的概率进入了不同的分支。

4.10

传统的决策树在进行节点划分时通常每次只选择一个属性，分类边界与坐标轴平行。而多变量决策树则使用多个属性的线性组合，分类边界可以是斜的超平面。

下面是一个多变量决策树的程序代码

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.linear_model import LogisticRegression

```

```

4
5 # 1. 加载并处理西瓜数据集3.0
6 data = {
7     '色泽': ['青绿', '乌黑', '乌黑', '青绿', '浅白', '青绿', '乌黑', '乌黑', '乌黑', '青绿', '浅白', '浅
    ↳ 白', '青绿', '浅白', '乌黑', '浅白', '青绿'],
8     '根蒂': ['蜷缩', '蜷缩', '蜷缩', '蜷缩', '蜷缩', '稍蜷', '稍蜷', '稍蜷', '稍蜷', '硬挺', '硬挺', '蜷
    ↳ 缩', '稍蜷', '稍蜷', '稍蜷', '蜷缩', '蜷缩'],
9     '敲声': ['浊响', '沉闷', '浊响', '沉闷', '浊响', '浊响', '浊响', '浊响', '沉闷', '清脆', '清脆', '浊
    ↳ 响', '浊响', '沉闷', '浊响', '浊响', '沉闷'],
10    '纹理': ['清晰', '清晰', '清晰', '清晰', '清晰', '清晰', '清晰', '稍糊', '清晰', '稍糊', '清晰', '模糊', '模
    ↳ 糊', '稍糊', '稍糊', '清晰', '模糊', '稍糊'],
11    '脐部': ['凹陷', '凹陷', '凹陷', '凹陷', '凹陷', '稍凹', '稍凹', '稍凹', '稍凹', '平坦', '平坦', '平
    ↳ 坦', '凹陷', '凹陷', '稍凹', '平坦', '稍凹'],
12    '触感': ['硬滑', '硬滑', '硬滑', '硬滑', '硬滑', '软粘', '软粘', '硬滑', '硬滑', '软粘', '硬滑', '软
    ↳ 粘', '硬滑', '硬滑', '软粘', '硬滑', '硬滑'],
13    '密度':
    ↳ [0.697, 0.774, 0.634, 0.608, 0.556, 0.403, 0.481, 0.437, 0.666, 0.243, 0.245, 0.343, 0.639, 0.657,
14    0.360, 0.593, 0.719],
15    '含糖率':
    ↳ [0.460, 0.376, 0.264, 0.318, 0.215, 0.237, 0.149, 0.211, 0.091, 0.267, 0.057, 0.099, 0.161, 0.198,
16    0.370, 0.042, 0.103],
17    '好瓜':
    ↳ ['是', '是', '是', '是', '是', '是', '是', '是', '是', '否', '否', '否', '否', '否', '否', '否', '否', '否']
18 }
19
20 df = pd.DataFrame(data)
21 # 将目标变量转为 0 和 1
22 y = df['好瓜'].map({'是': 1, '否': 0}).values
23 # 将离散特征进行 One-Hot 编码
24 X_df = pd.get_dummies(df.drop('好瓜', axis=1), dtype=float)
25 X = X_df.values
26 feature_names = X_df.columns
27 # 2. 定义多变量决策树节点与构造函数
28 class MultiVarNode:
29     def __init__(self):
30         self.is_leaf = False
31         self.label = None
32         self.model = None # 存放线性分类器
33         self.left = None # 预测为0的分支
34         self.right = None # 预测为1的分支
35
36 def build_multivar_tree(X, y, depth=0):
37     node = MultiVarNode()
38     # 终止条件1: 纯节点
39     if len(np.unique(y)) == 1:
40         node.is_leaf = True
41         node.label = y[0]
42         return node

```

```

43
44     # 训练多变量线性模型（无正则化以尽可能拟合）
45     model = LogisticRegression(penalty=None, solver='lbfgs', max_iter=1000)
46     model.fit(X, y)
47     preds = model.predict(X)
48
49     # 终止条件2: 模型无法进一步区分样本（预测全为一类）
50     if len(np.unique(preds)) == 1:
51         node.is_leaf = True
52         node.label = np.bincount(y).argmax() # 返回多数类
53         return node
54
55     # 记录模型并继续分裂
56     node.model = model
57     mask_left = (preds == 0)
58     mask_right = (preds == 1)
59
60     node.left = build_multivar_tree(X[mask_left], y[mask_left], depth+1)
61     node.right = build_multivar_tree(X[mask_right], y[mask_right], depth+1)
62
63     return node
64
65 # 3. 打印多变量决策树
66 def print_tree(node, depth=0):
67     indent = " " * depth
68     if node.is_leaf:
69         label_str = "好瓜(是)" if node.label == 1 else "好瓜(否)"
70         print(f"{indent}---> [叶子节点]: 分类结果 = {label_str}")
71         return
72
73     # 获取线性模型的权重和截距
74     weights = node.model.coef_[0]
75     intercept = node.model.intercept_[0]
76
77     #构造多变量方程字符串（只展示权重绝对值大于0.5的特征，方便观看）
78     equation = []
79     for w, feat in zip(weights, feature_names):
80         if abs(w) > 0.5:
81             equation.append(f"({w:.2f} * {feat})")
82     eq_str = " + ".join(equation) + f" + ({intercept:.2f}) > 0"
83
84     print(f"{indent}[内部节点]: 判断 {eq_str}")
85     print(f"{indent}如果为 False (预测0):")
86     print_tree(node.left, depth + 1)
87     print(f"{indent}如果为 True (预测1):")
88     print_tree(node.right, depth + 1)
89
90 # 4. 运行并观察结果

```

```

91 tree = build_multivar_tree(X, y)
92 print("===== 多变量决策树结构 =====")
93 print_tree(tree)
94

```

```

1      ===== 多变量决策树结构 =====
2      [内部节点]: 判断  $(-72.46 * \text{密度}) + (-751.91 * \text{含糖率}) + (38.96 * \text{色泽\_乌黑}) + (-118.59 * \text{色泽\_浅白}) + (-14.78 * \text{色泽\_青绿}) + (-223.57 * \text{根蒂\_硬挺}) + (9.72 * \text{根蒂\_稍蜷}) + (119.45 * \text{根蒂\_蜷缩}) + (109.73 * \text{敲声\_沉闷}) + (19.44 * \text{敲声\_浊响}) + (-223.57 * \text{敲声\_清脆}) + (-315.25 * \text{纹理\_模糊}) + (179.88 * \text{纹理\_清晰}) + (40.97 * \text{纹理\_稍糊}) + (335.65 * \text{脐部\_凹陷}) + (-526.51 * \text{脐部\_平坦}) + (96.46 * \text{脐部\_稍凹}) + (-139.12 * \text{触感\_硬滑}) + (44.72 * \text{触感\_软粘}) + (-94.40) > 0$ 
3      如果为 False (预测0):
4      ---> [叶子节点]: 分类结果 = 好瓜(否)
5      如果为 True (预测1):
6      ---> [叶子节点]: 分类结果 = 好瓜(是)

```

也就是说，通过将值代入上述式子，看式子结果的正负可以判断是好瓜还是坏瓜。这里如果为正则分类为好瓜，

此外我们观察式子中的系数，为正通常代表此项越高越有可能是好瓜。当然也会存在过拟合的情况，以及因为数据集太小了导致的误差。